

INE5425 · MODELAGEM E SIMULAÇÃO · UFSC · 2026

Autômatos Celulares Universais no GenESyS

Extensão do suporte a autômatos celulares no GenESyS, com configuração de reticulado, vizinhança, contorno, conjunto de estados, regra local e política de atualização.

João Vitor Curcio Sutter · Pedro Henrique Gimenez

Desenvolvimento de Componente de Simulação · Tema 6.2

CONTEXT0

O que é um autômato celular

01 **Lattice** · a malha de células e suas coordenadas

02 **Conjunto de estados** · valores possíveis de cada célula

03 **Vizinhança** · quais células influenciam cada célula

04 **Regra local** · o próximo estado a partir do estado atual e dos vizinhos

05 **Política de atualização** · quando e como as células avançam

O GenESyS já trazia uma base para isso, limitada a poucos casos fixos no código.

OBJETIVO

Generalizar o componente existente

Estender o `CellularAutomataComp` para representar autômatos síncronos e assíncronos, uniformes e não uniformes, de uma a N dimensões e em diferentes topologias de reticulado.

O ponto central do tema: a **regra local definida pelo usuário**.

ARQUITETURA

Quatro camadas

01 · ESPACIAL

Lattice

Coordenadas N-dimensionais,
conversão índice/posição.

Lattice · Cell · State

02 · RELACIONAL

Vizinhança e contorno

Quem são os vizinhos e o que acontece
nas bordas.

Neighborhood · BoundaryCondition

03 · SEMÂNTICA

Estados e regra

O que é um estado válido e como ele
transiciona.

StateSet · LocalRule

04 · TEMPORAL

Motor e política

Um passo do autômato e a ordem de
atualização.

CellularAutomataBase

Diagramas de classes e de sequência completos acompanham a entrega.

CONFIGURAÇÃO

Parâmetros configuráveis

RETICULADO

- Reticular
- Triangular
- Hexagonal
- Network

CONTORNO

- Fixed
- Closed
- Reflexive
- Adiabatic

REGRA LOCAL

- Elementary
- Game of Life
- Growty
- UserDefined

VIZINHANÇA

- Center
- Moore
- Von Neumann
- Triangular
- Hexagonal

CONJUNTO DE ESTADOS

- Enumerated
- Integer
- Bit
- Double

POLÍTICA DE ATUALIZAÇÃO

- Synchronous
- Sequential
- Random
- Blocks

Regra local definida pelo usuário

```
ca->setUserDefinedRuleSource(  
    "extern \"C\" long nextState("  
    "    long self, const long* n, int k) {"  
    "    return n[0] ^ n[1];"    // regra 90  
    "});"  
ca->setLocalRuleType(LocalRuleType::USERDEFINED);
```

O usuário escreve a transição em C++. O componente compila em tempo de execução via [CppCompiler](#), carrega a biblioteca dinâmica e a invoca por célula a cada passo.

```
nextState(self, n, k)
```

Estado atual, vizinhos e quantidade. Contrato simples.

```
nextStateEx(..., position, dims)
```

Acrescenta a posição N-dimensional da célula. Regras sensíveis à coordenada.

ATUALIZAÇÃO TEMPORAL

Política de atualização explícita

SYNCHRONOUS

Simultânea

Todas as células avançam no mesmo passo, em duas fases.

SEQUENTIAL

Sequencial

Atualização determinística, célula a célula.

RANDOM

Aleatória

Ordem pseudoaleatória com semente fixa. Reprodutível.

BLOCKS

Em blocos

Atualização por blocos de tamanho configurável.

PERSISTÊNCIA E VERIFICAÇÃO

Persistência e checagem semântica

`_saveInstance` / `_loadInstance`

Configuração preservada

Tipo de autômato, reticulado, vizinhança, contorno, estados, regra, dimensões, arestas de rede e o código fonte da regra do usuário.

`_check`

Coerência validada

Lattice contra vizinhança, estados contra regra, viabilidade da política e compilação da regra do usuário. Configuração inválida retorna erro sem derrubar o simulador.

VALIDAÇÃO

Resultado dos testes

47/47

testes do componente, todos
aprovados, em ambiente Linux
limpo (Docker).

14 neighborhood

11 cellular_automata

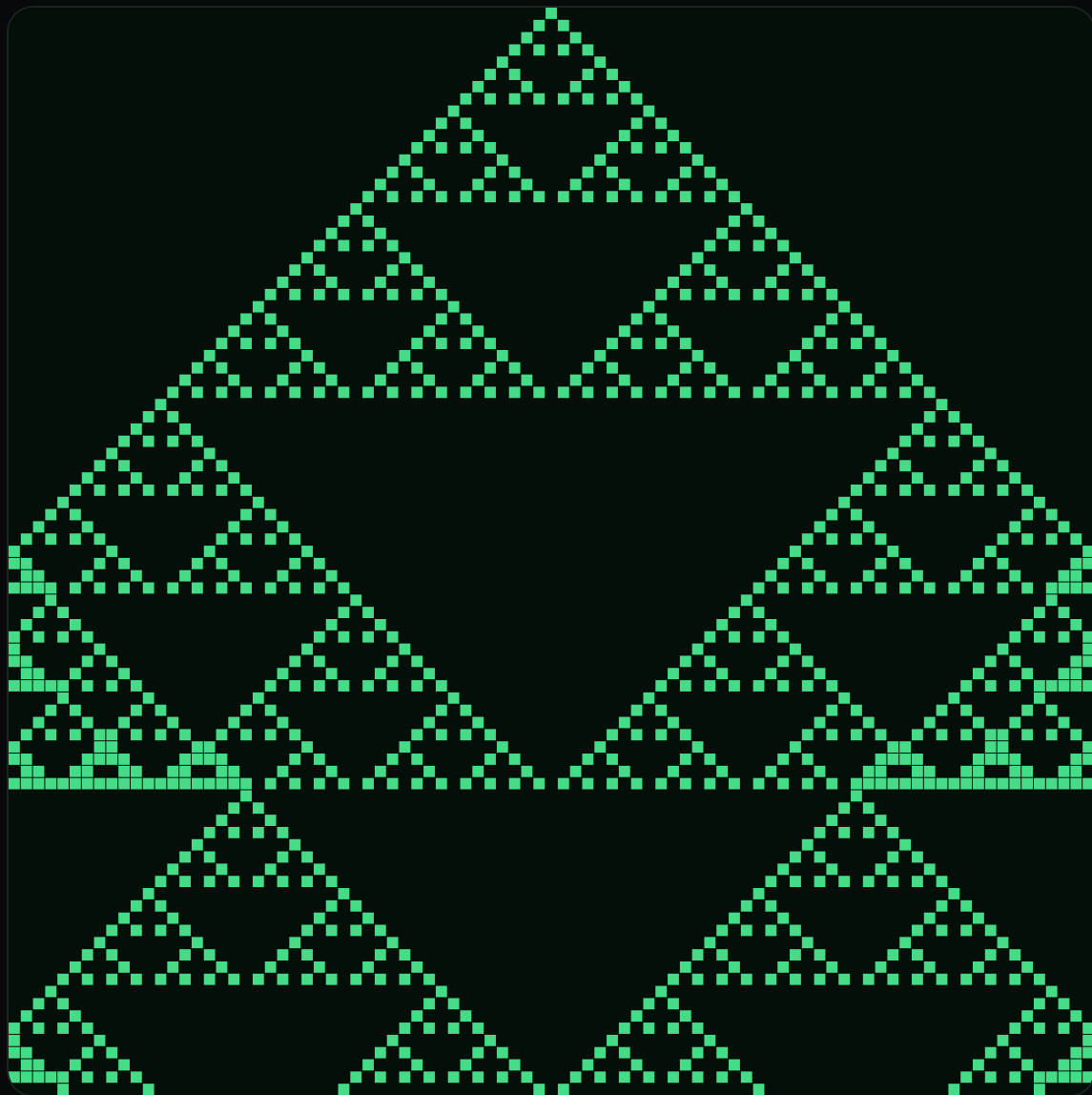
10 component

8 user_defined

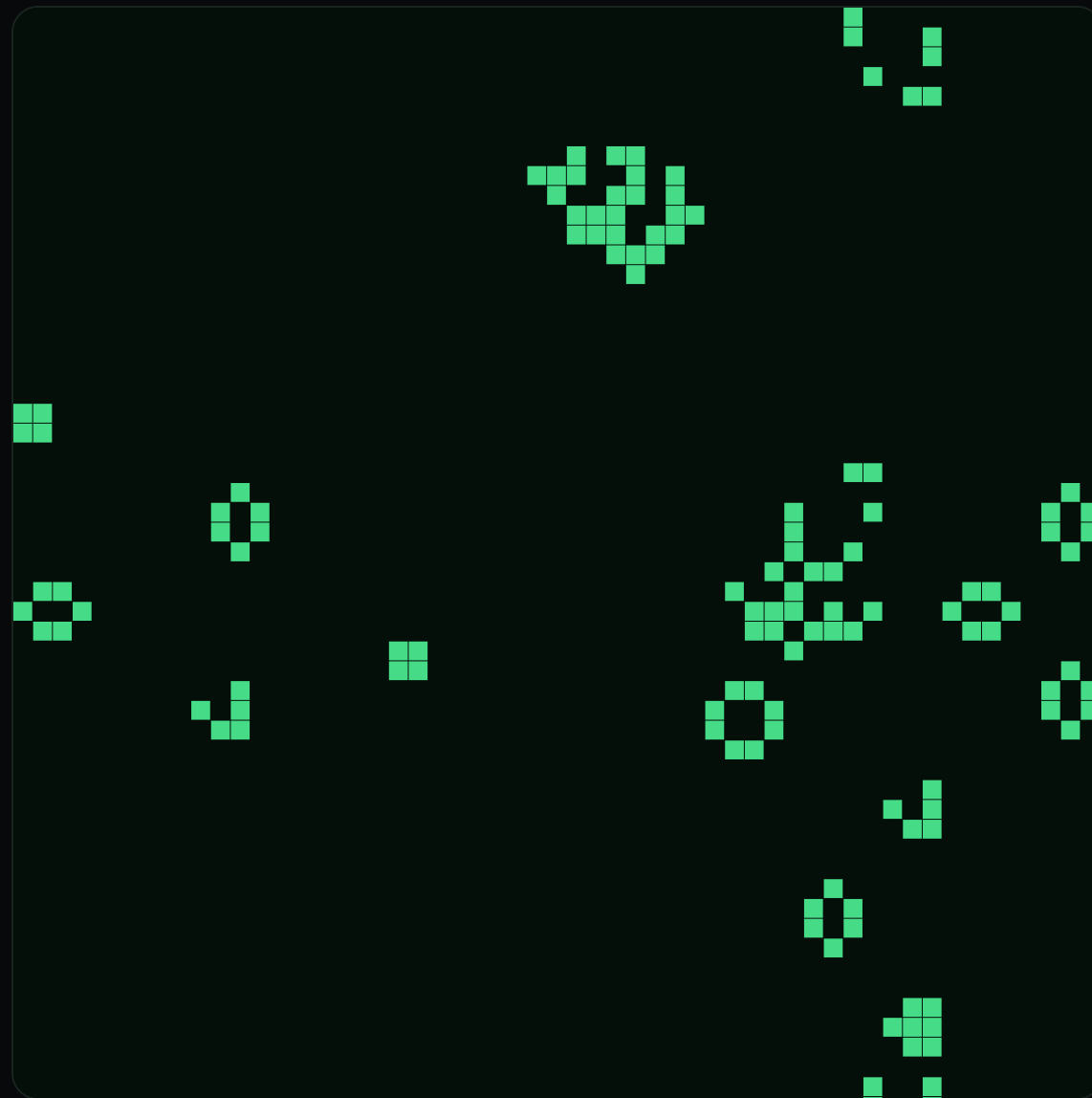
4 elementary

Reprodução exata das referências teóricas: regra 30, regra 90 (triângulo de Sierpinski) e Game of Life (blinker, glider).

O autômato em execução



Regra 90 · triângulo de Sierpinski a partir de uma célula



Game of Life · B3/S23 com gliders

EQUIPE

Divisão do trabalho

Generalização universal (infraestrutura)

João Vitor Curcio Sutter

Reticulados e topologias (triangular, hexagonal, network), vizinhanças, condições de contorno e o exemplo de terminal da regra 90.

Regra do usuário (Linha B) e integração

Pedro Henrique Gimenez

Compilação dinâmica via CppCompiler, classe LocalRule_UserDefined, integração ao componente, checagem e persistência.

CONCLUSÃO

Componente generalizado, integrado e **verificado**

Suporte a múltiplas topologias, condições de contorno, conjuntos de estados e regras locais, incluindo regra definida pelo usuário compilada em C++. O componente segue a arquitetura do GenESyS e preserva a persistência de modelos.

PR `JVSutter:tema6-currentStable` → `rlcancian:currentStable`

João Vitor Curcio Sutter Pedro Henrique Gimenez